Cybersecurity Case Studies & Solutions

Comprehensive Analysis, Vulnerability Assessments, and Risk Mitigations

Case Study 1: Legacy Windows RPC Vulnerabilities & Patch Management

1. Exploitation & Privilege Escalation Pathways

- **Reconnaissance & Enumeration:** Attackers scan TCP port 135 (RPC Endpoint Mapper) using tools like rpcdump or impacket to enumerate active interfaces, registered endpoints, and exposed UUIDs.
- **Remote Code Execution (RCE):** Legacy vulnerabilities (e.g., MS08-067, PetitPotam, or PrintNightmare via MS-RPRN) allow unauthenticated buffer overflows or protocol coercion to execute arbitrary shellcode remotely.
- **Privilege Escalation:** RPC services natively execute under high-privilege system contexts (NT AUTHORITY\SYSTEM or LOCAL SERVICE). Successful exploitation grants instant administrative compromise without secondary escalation steps.
- **Lateral Movement:** Attackers extract credentials from LSASS, harvest Kerberos tickets, and pivot laterally across internal financial subnets.

2. Risk Assessment of Delayed Patching

- **Risk Level:** CRITICAL. In a financial institution, exposed RPC services directly threaten transactional databases, sensitive customer records, and regulatory compliance (PCI-DSS, SOX).

3. Strategy Justification: Immediate vs. Controlled Maintenance

> **Strategic Justification:** Hybrid Rapid Deployment (Canary Model) is strongly justified over standard maintenance cycles. Abnormal network traffic indicates active exploitation attempts. Standard scheduling creates an unacceptable threat window. Immediate deployment must include a rapid 2–4 hour staging validation followed by canary rollout to prevent system bricking while eliminating exposure within 24 hours.

4. Recommended Mitigation Strategies (Zero Downtime)

- **Network Micro-Segmentation:** Block external/untrusted access to TCP port 135 and dynamic RPC ports (49152–65535), restricting access strictly to authorized jump servers.
- **RPC Endpoint Filtering:** Configure host-level RPC filtering rules to restrict unauthenticated access to sensitive RPC interfaces.
- **Virtual Patching / IPS:** Deploy deep-packet inspection IPS signatures on firewalls to detect and drop RPC exploit payloads before reaching hosts.
- **Credential Safeguards:** Enforce strict Least Privilege and disable Domain Admin interactive logins on legacy member servers.

Case Study 2: Web App Vulnerabilities (XSS, CSRF & Weak Session Handling)

1. Attack Chaining & Exploitation Mechanism

- **Chained Session Hijacking:** Attackers inject persistent Stored XSS into user inputs. When an authenticated user loads the page, the script steals document.cookie and exfiltrates session tokens.
- **XSS-Bypassed CSRF Exploitation:** Because script executes within the victim's authenticated origin, the script dynamically reads anti-CSRF tokens from the DOM and sends forged financial transactions.
- **Predictable Session Tokens:** Low-entropy session generation allows attackers to brute-force or guess active session identifiers, while missing logout invalidation enables indefinite reuse.

2. Risk Assessment

- **Business & Financial Impact:** Direct financial losses through fraudulent wire transfers, account takeover (ATO), and severe regulatory fines under GDPR / PCI-DSS.

3. Justification for Layered Security (Defense-in-Depth)

> **Defense-in-Depth Justification:** Single-point defenses fail when vulnerabilities chain together. Relying solely on anti-CSRF tokens fails against XSS; relying only on session timeouts fails against active token theft. Multiple overlapping controls ensure that the failure of one defense does not compromise the application.

4. Comprehensive Mitigation Strategies

- **Input Sanitization & Context-Aware Encoding:** Enforce strict allow-lists and context-aware output encoding (HTML, JS, CSS) using libraries like OWASP Java Encoder or DOMPurify.
- **Content Security Policy (CSP):** Implement strict CSP headers (e.g., Content-Security-Policy: default-src 'self'; script-src 'nonce-xxx';) to block unauthorized script execution.
- **Secure Cookie & Session Hardening:** Set cookie flags: HttpOnly (blocks XSS access), Secure (HTTPS only), and SameSite=Strict (mitigates CSRF). Enforce 128-bit session entropy and 5–10 min idle timeouts.
- **Step-up Re-Authentication:** Mandate MFA or password re-prompting prior to high-risk actions (e.g., funds transfer, password change).

Case Study 3: Outdated IIS Web Server & High-Availability Patching

1. Attack Vectors on Legacy IIS

- **Remote Code Execution (RCE):** Outdated IIS versions are vulnerable to kernel/driver-level flaws (such as HTTP.sys CVE-2015-1635 / MS15-034) or buffer overflows in WebDAV modules, enabling complete server takeover via w3wp.exe.

- **Directory Traversal & Info Leaks:** Exploits allow attackers to bypass URL filters, dump source code, and steal database connection strings from web.config.
- **Web Shell Deployment:** Attackers drop persistent ASPX web shells into application directories, converting the web server into a command-and-control pivot.

2. Strategy Justification: Immediate Patching vs. Temporary Isolation

> **Strategic Justification:** Temporary Shielding & Isolation with Blue-Green Deployment is recommended. Direct unvetted patching risks breaking critical legacy dependencies. Placing the IIS instance behind a Reverse Proxy / WAF provides immediate exploit shielding (virtual patching) while allowing updates to be deployed seamlessly with zero downtime.

3. Recommended Availability & Security Controls

- **WAF Virtual Patching:** Deploy WAF rules to inspect HTTP request headers and drop known exploit patterns targeting IIS/HTTP.sys.
- **Blue-Green Deployment:** Deploy an identical staging environment, apply updates, validate application compatibility, and switch production traffic via load balancing with zero downtime.
- **Application Pool Isolation:** Run IIS application pools under dedicated Group Managed Service Accounts (gMSA) with minimal local rights; disable unnecessary features like WebDAV.

Case Study 4: Insecure Mobile App Storage & Unencrypted Communications

1. Exploitation Vectors

- **Local Storage Extraction:** Plaintext data stored in SharedPreferences, NSUserDefaults, or unencrypted SQLite databases can be extracted via ADB backup or file access on rooted/jailbroken devices.
- **Man-in-the-Middle (MitM) Interception:** Cleartext HTTP traffic over public Wi-Fi allows packet capture of login credentials, session tokens, and PII using tools like Wireshark or mitmproxy.

2. Cryptographic Security Standards

Domain	Vulnerable Practice	Secure Implementation Standard
Data at Rest	Plaintext files, standard SQLite, SharedPreferences.	Android Keystore + EncryptedSharedPreferences / SQLCipher (AES-256-GCM). iOS Keychain Services API.
Data in Transit	Plaintext HTTP, ignoring SSL/TLS certificate errors.	Enforce HTTPS with TLS 1.3 across all endpoints. Strict Network Security Config / App Transport Security.
MitM Defense	Relying strictly on default OS Certificate Authorities.	Implement SSL / Certificate Pinning (Public Key Hash Pinning) inside the mobile binary.

Runtime Security	Running unmodified on rooted/jailbroken environments.	Implement root/jailbreak detection, screen-capture blocking, and disable OS backup (android:allowBackup="false").

Case Study 5: Firewall Mismanagement & Automated Policy Enforcement

1. Exploitation of Temporary Firewall Exposure

- **Port Scanning & Service Discovery:** Automated bots and internal threat actors detect open ports (RDP 3389, SMB 445, WinRM 5985, RPC 135) within seconds of firewall deactivation.
- **Brute-Force & Credential Spraying:** Unprotected management interfaces become targets for rapid authentication attacks.
- **Direct Exploitation:** Internal services that lack application-level hardening are directly exploited via network-level buffer overflows.

2. Risks of Security Control Mismanagement

- **Configuration Drift & Silent Persistence:** Admins forget to restore security controls, allowing attackers to establish persistent backdoors (scheduled tasks, hidden accounts) that survive even after manual firewall reactivation.

3. Importance of Automated Policy Enforcement

> **Automation Justification:** Manual security controls rely on human discipline, making them vulnerable to human error. Automated policy enforcement continuously evaluates the desired security baseline and automatically reverts unauthorized modifications without human intervention.

4. Recommended Administrative Safeguards

- **Centralized Baseline Enforcement:** Enforce Windows Defender Firewall configurations via Active Directory Group Policy (GPO) or Intune MDM, preventing local disabling.
- **Self-Healing Configuration Scripts:** Deploy configuration management (PowerShell DSC, Ansible) to automatically re-enable disabled firewall profiles on a recurring schedule.
- **Privileged Access Management (PAM):** Implement Just-In-Time (JIT) access and remove permanent local administrative rights.
- **SIEM Alerting & Monitoring:** Set high-priority alerts for Windows Security Event ID 5025 ('Firewall Service stopped') and Event ID 4950 ('Firewall setting changed').
- **Standardized Troubleshooting Protocols:** Mandate testing via targeted port-opening rules or dedicated sandbox lab environments rather than disabling global host firewalls.